

TABLE OF CONTENTS

1. INTRODUCTION	5
1.1 Purpose and Scope	5
1.2 Problem Statement	5
1.3 Solution Statement	5
1.4 Contribution	6
1.5 Glossary	6
2. LITERATURE REVIEW	8
2.1 The Planning Fallacy and Static Scheduling Systems	8
2.2 Cognitive Fatigue and Academic Burnout Prediction	8
2.3 Machine Learning in Educational Data Mining (EDM)	8
2.4 Large Language Models (LLMs) in Adaptive Education	8
2.5 Summary and Identified Gap in the Literature	9
3. SOFTWARE REQUIREMENTS SPECIFICATION	10
3.1 Introduction	10
3.2 The Overall Description	10
3.2.1 Product Perspective	10
3.2.1.1 System Interfaces	10
3.2.1.2 Interfaces (User Interfaces)	10
3.2.1.3 Hardware Interfaces	11
3.2.1.4 Software Interfaces	11
3.2.1.5 Communications Interfaces	11

3.2.1.6 Memory Constraints	11
3.2.1.7 Operations	11
3.2.1.8 Site Adaptation Requirements	11
3.3 Product Functions	12
3.4 User Characteristics	12
3.5 Constraints	12
3.6 Assumptions and Dependencies	12
3.7 Apportioning of Requirements	13
3.8 Specific Requirements	13
3.8.1 Functions	13
3.8.2 Performance Requirements	14
3.8.3 Logical Database Requirements	14
3.8.4 Design Constraints	15
3.8.4.1 Standards Compliance	15
3.8.5 Software System Attributes	15
3.8.5.1 Reliability	15
3.8.5.2 Availability	15
3.8.5.3 Security	15
3.8.5.4 Maintainability	15
3.8.5.5 Portability	15
3.8.6 Organizing the Specific Requirements	15
4. SYSTEM DESIGN SPECIFICATION	16

4.1 Introduction	16
4.2 SYSTEM ARCHITECTURE	16
4.2.1 System Software Architecture	16
4.3 DATABASE DESIGN	17
4.3.1 Database Overview	17
4.3.2 Narrative Data Description	17
4.3.3 Entity Relationship Diagram (ERD)	17
4.3.4 Data Integrity and Constraints	17
4.4 HUMAN-MACHINE INTERFACE	17
4.4.1 Functional Usage Description	21
4.4.2 Inputs	21
4.4.3 Outputs	22
4.5 DETAILED DESIGN	22
4.5.1 Software Detailed Design	22
5. IMPLEMENTATION	24
5.1 Overview of the AI Engine Implementation	24
5.2 Core Components and Logic	24
5.2.1 Server Initialization and Routing	24
5.2.2 Intelligence Layer (Gemini Integration)	24
5.3 Functional Proof and End-to-End Testing	25
5.4 Implementation Robustness	26
6. RESULT & CONCLUSION	27

6.1 Results and Discussion	27
6.1.1 Testing and Validation	27
6.1.2 What was Learned	27
6.2 Conclusion	27
6.3 Future Work and Improvements	28
REFERENCES	29

1. INTRODUCTION

1.1 Purpose and Scope

The final version of Smart Study Planner is now a working study planning system, not only a project idea. The application lets the student register with email and password, complete onboarding, add courses, create tasks from courses or calendar, generate a daily plan, start focus sessions, and review simple analytics about real progress.

The scope became wider than the mid term prototype, but the main concept stayed the same: the planner should reduce confusion for the student and help him understand what to study today. The final application combines manual control with optional AI, so the student can depend on the normal scheduler first and then ask the AI for a smarter second pass when it is needed.

The final implementation includes a React Native Expo front end, an ASP.NET Core backend, SQLite persistence for the temporary deployment, and a Python FastAPI AI service. This structure makes the project practical for mobile and web use, and it keeps the AI logic separate from the main API. The student sees one simple app, while behind it there are three services working together.

In the final project, the application is centered around courses, which are the subjects that a student studies in the semester. Examples are Calculus II, Physics Lab, or Academic Writing. Each course can have a difficulty level, a priority, and optional midterm and final dates. If a course has no midterm or final, the system still accepts it, because not every course is exam based.

The scope also includes a calendar flow. The student can open a date, see the tasks for that day, add a new task, and see the daily program generated from the available slots. Completed tasks are shown as done, and the application gives a small positive message with the number of remaining tasks. This is important because the student should feel progress, not only pressure.

1.2 Problem Statement

Most students do not fail because they do not have a calendar. They fail because the calendar is too static and it does not understand how studying actually happens. A student may plan Calculus for one hour, but the task takes longer. He may start one course, finish half a session, and then move to another course. A normal planner does not know what this means.

The main problem is that students need a planner that can show what is already done, what is still open, and what needs attention inside each course. The student also needs to know if he is studying in a balanced way or if all effort is going to one course while another course is ignored. This gap was very clear in the final stage when the project moved from screens to a real working application.

Another problem is the first experience after registration. If onboarding is confusing, the student may avoid the important setup and the system cannot help him. In the final version we made the first setup more direct: the student enters a first course, optional exam dates, an initial task, and study time slots. This data gives the system enough context to start helpful scheduling.

1.3 Solution Statement

The final solution is a student planner that connects courses, tasks, calendar, focus sessions, and analytics in one flow. The student starts from onboarding, adds courses with difficulty and optional exams, then creates tasks. The calendar can show task work by date and create new tasks from the selected day. The Focus page records actual study minutes and focus rating.

The schedule is generated in two possible ways. First, the backend can build a rule based plan using priority, difficulty, deadlines, available slots, and time blocks. Second, the student can ask for AI improvement. This is safer for the user because the app works even if the AI service is not used. Therefore the AI is powerful, but it is not the only way the app can function.

The analytics are intentionally simple. Instead of complex dashboards, the final UI shows a Study Pulse: daily streak, done tasks, open tasks, today study time, focus rating, weekly rhythm, achievements, and course momentum. The point is to wake the student to details he may not notice, like one course being behind or a streak being close to the next milestone.

1.4 Contribution

This project contributes a complete and connected study workflow. It is not only a to-do list and it is not only an AI demo. The application stores real student data, uses a backend API, creates schedules, and displays progress in a mobile friendly interface.

The contribution is also in the user experience. Course setup supports midterm, final, or no exam dates. Calendar tasks can be created for specific days. Focus sessions can be attached to a course and optionally to a task. Analytics show progress without making the student feel that he entered a strict boot camp.

- Email/password authentication with JWT access tokens and refresh tokens.
- Course management with difficulty, priority, midterm date, final date, and no-exam support.
- Task management from course detail and calendar, including completed task tracking.
- Rule based schedule generation with optional AI improvement through the FastAPI AI service.
- Focus sessions that record actual study time, focus rating, and partial work on tasks.
- Study Pulse analytics for streak, achievements, course momentum, and weekly rhythm.
- Temporary HTTPS deployment using sslip.io, Caddy reverse proxy, Docker Compose, and GitHub Actions.

1.5 Glossary

- **Course / Subject:** The academic course that the student studies, such as Calculus or Physics. In the code it is also called Subject.
- **Task:** A concrete study item connected to one course, with title, estimated minutes, difficulty, priority, optional deadline, and status.
- **Available Slot:** A time range where the student is free to study. The scheduler uses these slots to place tasks.
- **Rule Based Plan:** A schedule generated by backend logic without calling the AI service. It is the first working plan.

- **AI Improvement:** An optional call to the Python AI engine and Gemini service to produce a more intelligent schedule.
- **Burnout Score:** A value from 0.0 to 1.0 calculated from snoozes, study hours, and focus rating.
- **Difficulty Factor:** A ratio that compares actual time with estimated time, so harder courses can get more time later.
- **Focus Session:** A timer based study session connected to a course and optionally to a task. It stores actual time and rating.
- **Study Pulse:** The final analytics screen showing streaks, wins, study hours, and course momentum.

2. LITERATURE REVIEW

The Smart Study Planner is based on ideas from time management, cognitive psychology, Educational Data Mining, and modern AI systems. The final report keeps the same research direction as the mid term report, but the discussion is now connected to the features that were really implemented.

2.1 The Planning Fallacy and Static Scheduling Systems

The planning fallacy explains why people often underestimate the time needed to finish a task. This is very visible in studying. A student may think that one exercise sheet needs thirty minutes, but after starting it he discovers that it needs one hour or more. Static scheduling systems accept the first guess and do not learn from the actual time.

The final system answers this problem by storing estimated minutes and actual minutes. When a task is completed, the backend saves actualMinutes. The analytics can then calculate planning error minutes. In the demo data, the system showed a small negative planning error because two completed tasks finished slightly faster than estimated. This makes the feedback real, even when the conclusion looks small at first.

2.2 Cognitive Fatigue and Academic Burnout Prediction

Academic fatigue is not only about the number of tasks. It also depends on focus quality, study hours, and how often the student delays work. Research about burnout usually explains that exhaustion grows when work continues without balance and without rest.

The final AI engine calculates a burnout score using snooze count, study hours, and average focus. If focus is low, the score becomes higher. If focus is strong, the score can be reduced slightly. This is simple, but it is suitable for a final student project because the logic is understandable and can be tested. Therefore the risk is not magic, it is a calculated signal.

2.3 Machine Learning in Educational Data Mining (EDM)

Educational Data Mining uses student behavior data to improve learning decisions. In this project the data is not grades only. It includes completed tasks, actual minutes, focus sessions, snoozes, and weekly study hours.

The AI engine starts with heuristics because a new user has no history. When enough completed task data exists, the code can move toward personalized models. The model manager uses regression for subject difficulty and a burnout model for exhaustion risk. This is close to the literature because it learns from behavior, not from a fixed schedule.

2.4 Large Language Models (LLMs) in Adaptive Education

LLMs can be useful in education when they are used with a clear structure. In this project the LLM is not used as a general chatbot. It receives structured data from the backend and returns a structured JSON schedule. The AI prompt asks for time slots, breaks, postponed tasks, and a student friendly message.

The final application also supports non-AI generation. This point is important because an educational tool should not stop if an AI key is missing or the service is slow. The AI remains a second option for improvement, while the normal plan remains available for daily use.

2.5 Summary and Identified Gap in the Literature

The gap is between static planners and truly adaptive study support. Many tools can store events, but fewer tools combine task difficulty, actual time, focus rating, burnout risk, and a simple student interface. The final Smart Study Planner tries to fill this gap in a practical way.

The project does not claim to replace teachers or full academic advising. It is more like a quiet study assistant. It helps the student see today's work, understand course progress, and keep a healthy rhythm. This was the main design direction in the final stage.

3. SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Introduction

The Software Requirements Specification describes what the final system must do and how the parts must behave. The final project has three main services: mobile/web front end, backend API, and AI engine. The requirements below are based on the implemented code and the final deployed application.

The student must be able to use the application without confusion. The system must support adding courses, adding tasks, selecting calendar dates, starting focus sessions, and seeing progress. It must also keep user data private through authenticated API calls.

3.2 The Overall Description

The system is a personal academic planning application. It is used by one student account at a time. The backend stores users, courses, tasks, focus sessions, available slots, behavioral logs, and generated schedules. The front end presents these data in a mobile style interface.

3.2.1 Product Perspective

Smart Study Planner is independent from the Lisan project on the temporary server. It is deployed under a TMP directory and exposed through sslip.io subdomains. If the TMP folder is removed later, it should not affect the existing Lisan project.

The final deployment uses Docker Compose services and Caddy reverse proxy. The front end, backend, and AI service are connected inside the Docker network. GitHub Actions can sync the repository to the server and run the deployment script.

3.2.1.1 System Interfaces

The front end communicates only with the backend API. The backend communicates with the AI service only when the schedule is generated with the AI option. The database is accessed only by the backend through Entity Framework Core.

Interface	Implemented detail
Frontend to Backend	HTTPS REST API under /api/v1 with JSON payloads.
Backend to AI	HTTP client calls FastAPI endpoint /api/v1/optimize-schedule when useAi is true.
Backend to DB	Entity Framework Core with SQLite in the temporary deployment.
Deployment	Caddy routes smart-study-project and api.smart-study-project sslip.io hosts.

3.2.1.2 Interfaces (User Interfaces)

The user interface is built with React Native and Expo. It is designed for phone screens first, and it also works as a web export. The bottom navigation includes Home, Calendar, Courses, Focus, and Profile. The analytics page is opened from the Home header.

The main user screens are login, onboarding, dashboard, calendar, courses, course detail, focus timer, Study Pulse analytics, profile, edit profile, and change password. Google and Apple authentication are not part of the final authentication flow; the final login is email and password.

3.2.1.3 Hardware Interfaces

The mobile application is intended for Android and iOS devices through Expo, and it can also run as a web build in a modern browser. No special hardware sensors are required. Push notification support exists in the backend, but web notifications are limited by platform support.

3.2.1.4 Software Interfaces

The backend is built with ASP.NET Core 8. It uses ASP.NET Identity, JWT authentication, FluentValidation, Entity Framework Core, and health checks. The AI service is built with FastAPI, scikit-learn, joblib, and Gemini integration. The front end uses Expo SDK, React Native Web, Axios, AsyncStorage, charts, icons, and custom context providers.

3.2.1.5 Communications Interfaces

All client requests use HTTPS on the deployed version. The backend accepts JSON and returns JSON. CORS is configured for the Expo/web client. Internally, the backend uses HTTP to call the AI service by container name inside the Docker network.

3.2.1.6 Memory Constraints

The application is designed to stay light. The front end keeps only current user state, courses, tasks, behavior logs, and the latest schedule in memory. The backend stores durable data in SQLite. The AI service stores saved model files when it has enough training data, but it can still work in cold start mode without model history.

3.2.1.7 Operations

- Register or login with email and password.
- Complete onboarding with name, semester deadline, at least one course, and at least one study time slot.
- Create, update, and delete courses.
- Create, complete, snooze, update, and delete tasks.
- Generate a day plan from available slots, first without AI and optionally with AI.
- Start and complete focus sessions, including a rating and optional task finish.
- Read analytics and course progress.

3.2.1.8 Site Adaptation Requirements

For the final temporary deployment, the project is placed under /TMP/smart-study-planner on the server. The three public URLs are generated from sslip.io. This makes the deployment temporary and isolated from the other website on the same server.

The deployment script updates only the TMP smart study planner Caddy block. It builds the backend, AI service, and frontend containers, then reloads Caddy after validation. This supports a two or three day hosting solution while staying independent.

3.3 Product Functions

The product functions are grouped by student workflow. The final system begins with account creation, then onboarding, then daily use. The most important idea is that every screen should connect to another real action.

- Authentication: register, login, token refresh, logout, change password, and forgot password stub.
- Onboarding: collect basic student name, final semester deadline, first course, optional midterm/final dates, first task, and study slots.
- Courses: list courses, show open and done tasks, add/edit dates, set priority and difficulty, delete a course.
- Course detail: show course progress, finished tasks, open tasks, actual minutes, exam dates, and add new tasks.
- Calendar: month/week/day views, task dots, exam dates, add task for selected date, show done and remaining tasks, show generated daily program.
- Scheduling: create plan without AI from deadlines, priority, difficulty, and available slots; optionally improve with AI.
- Focus: choose course/task, start a timer, finish session, rate focus, switch course after partial work.
- Analytics: show streak, badges, weekly rhythm, focus rating, study time, and course momentum.

3.4 User Characteristics

The expected user is a student who has several courses and wants a simple way to decide what to study each day. The user may not understand complex project management concepts, so the interface uses words like course, task, done, open, focus, and streak.

The user may also be tired or stressed. For this reason, the application avoids heavy analytics and uses small motivating messages. The design target is not a strict productivity camp. It is a friendly academic tool that still records serious data.

3.5 Constraints

- The AI service needs an API key for Gemini to produce full LLM schedules.
- The app must still work without AI, so rule based scheduling is required.
- The temporary deployment uses SQLite, which is practical for a short server period but not ideal for a very large production system.
- The front end is mobile-first, so text and cards must fit within small screens.
- Real phone testing and web testing are both useful because the application is mobile-first but also exported to web.

3.6 Assumptions and Dependencies

The system assumes that every user has at least one course and at least one available study slot after onboarding. The backend validator enforces this, which means the planner has enough information to create a useful first plan.

The system also assumes that actual study time and focus rating are honest enough to be useful. If the user does not log real sessions, the analytics will be less accurate. So the system can help only after the student gives it real behavior, although the first result still appears in cold start.

3.7 Apportioning of Requirements

Component	Responsibility
Frontend	User screens, navigation, forms, focus timer, local token storage, and visual analytics.
Backend API	Authentication, authorization, validation, CRUD, schedule persistence, analytics, and AI client calls.
AI Engine	Burnout score, difficulty factors, Gemini schedule generation, and fallback heuristic output.
Deployment	Docker Compose, Caddy HTTPS routing, sslip.io hosts, and GitHub Actions deployment.

3.8 Specific Requirements

3.8.1 Functions

The final implementation satisfies the core functional requirements. The API exposes controllers for auth, users, subjects, tasks, focus sessions, available slots, schedules, analytics, and behavioral logs. Each controller uses the current authenticated user, so users cannot read each other's data.

- FR-1: The system shall allow a new user to register with name, email, and secure password.
- FR-2: The system shall require onboarding before entering the main app.
- FR-3: The system shall allow a course without midterm and without final.
- FR-4: The system shall allow a course with only final, only midterm, or both.
- FR-5: The system shall allow task creation from the course page and from the calendar page.
- FR-6: The system shall show completed tasks separately from open tasks.
- FR-7: The system shall generate a daily program from available slots.
- FR-8: The system shall record actual focus time and focus rating.
- FR-9: The system shall show simple analytics that make progress visible.

The functional requirements were implemented through separate API groups. This was helpful because each screen can request only the data that it needs. For example, Courses does not need to know about refresh tokens, and the Focus screen does not need to know about onboarding logic.

API group	Main implemented routes
Auth	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout, GET /auth/me.
Users	GET /users/me, PUT /users/me, POST /users/me/onboarding, POST /users/me/push-token.
Subjects	GET /subjects, POST /subjects, PUT /subjects/{id}, DELETE /subjects/{id}.
Tasks	GET /tasks, POST /tasks, PUT /tasks/{id}, POST /tasks/{id}/complete, POST /tasks/{id}/snooze.
Focus	GET /focus-sessions, POST /focus-sessions, PATCH /focus-sessions/{id}/complete.
Schedule	POST /schedule/generate, GET /schedule/today, GET /schedule/day/{date}, PATCH slot status.
Analytics	GET /analytics/insights and AI performance endpoint when metrics exist.

3.8.2 Performance Requirements

Normal CRUD actions should respond quickly because they use direct database operations. Schedule generation without AI should also be fast, because it is calculated in the backend. AI schedule improvement can take longer because it depends on the external model response, so the backend timeout is configured separately.

The deployed health check returned healthy for the database and AI service. This means the final server arrangement was able to run the three services together during final testing.

3.8.3 Logical Database Requirements

The logical database is based on users, courses, tasks, focus sessions, available slots, behavioral logs, refresh tokens, and generated schedules. The final migration added `midterm_date` and `final_date` to the subjects table. The old `exam_date` value is still kept as a fallback for compatibility.

Entity	Important fields
User	id, name, email, password hash, deadline, isOnboarded, pushToken.
Subject	id, userId, name, difficulty, priority, examDate, midtermDate, finalDate.
StudyTask	id, subjectId, title, priority, difficultyRating, estimatedMinutes, actualMinutes, status, deadline, tag.
FocusSession	id, userId, subjectId, taskId, mode, durationSeconds, focusRating, startedAt, completedAt.
AvailableSlot	id, userId, date or dayOfWeek, startTime, endTime.
AiSchedule	id, userId, generatedAt, mode, burnoutScore, requestPayload, responsePayload, slotStatusesJson.

3.8.4 Design Constraints

The UI must remain readable on a phone. Buttons must be large enough for touch. Text must not overlap cards or tab bars. The final screenshots show the mobile layout with bottom tabs and large readable cards. Some icons are used because it is faster for the student to scan them than reading long labels.

3.8.4.1 Standards Compliance

The backend follows common REST API structure and returns JSON. Authentication uses Bearer tokens. Validation is performed with FluentValidation. The front end uses Expo/React Native conventions, and the web export is served as static files behind nginx and Caddy.

3.8.5 Software System Attributes

3.8.5.1 Reliability

Reliability is supported by validation, health checks, and rule based scheduling. If AI fails or is not requested, the system can still produce a plan. This makes the main workflow less fragile.

3.8.5.2 Availability

The temporary deployment runs as Docker containers with restart policy unless-stopped. The application is exposed over HTTPS through Caddy. This gives enough availability for the final demo and short term public testing.

3.8.5.3 Security

Security is based on ASP.NET Identity password hashing, JWT access tokens, refresh tokens stored as hashes, and authorized API controllers. The final system does not use Google or Apple login. It uses email/password only, which keeps the final authentication flow simple.

3.8.5.4 Maintainability

Maintainability is supported by splitting the system into front end, application services, domain entities, infrastructure, API controllers, and the AI microservice. The final code is easier to improve because schedule generation, analytics, and AI are separated.

3.8.5.5 Portability

The front end can run on web, Android devices, iOS simulator, or phone through Expo. The backend and AI services can be moved because they are containerized. SQLite is enough for the temporary server, while EF Core makes a future database change possible.

3.8.6 Organizing the Specific Requirements

The requirements are organized around the student flow: account, onboarding, courses, calendar, plan, focus, analytics, and deployment. This ordering is used again in the design and implementation sections, so the final report remains connected and not random.

4. SYSTEM DESIGN SPECIFICATION

4.1 Introduction

The system design explains how the final application is built. The design changed from the early idea into a real multi-service project. The most important design rule is separation: the front end should not talk to the AI directly, and the AI should not own user accounts.

4.2 SYSTEM ARCHITECTURE

4.2.1 System Software Architecture

The final architecture has three runtime services. The Expo front end calls the ASP.NET Core API. The API stores data in SQLite and calls the FastAPI AI service only when needed. The AI service may call Gemini to generate the schedule, or it may return heuristic fallback output.

Layer	Main technology	Responsibility
Presentation	Expo React Native	Screens, forms, navigation, focus timer, and mobile user experience.
API	.NET 8 ASP.NET Core	Auth, validation, database logic, analytics, and schedule orchestration.
Persistence	EF Core + SQLite	Stores users, courses, tasks, slots, sessions, logs, and schedules.
AI Engine	Python FastAPI	Burnout score, difficulty factors, Gemini prompt, and fallback schedule.
Deployment	Docker + Caddy + GitHub Actions	Builds and serves the three services through HTTPS sslip.io domains.

The backend is the source of truth. This design prevents the front end from needing to know about model files or Gemini prompts. It also makes the AI replaceable later. If another model is used, the frontend still calls the same schedule endpoint.

The final deployment uses temporary HTTPS sslip.io hosts for the frontend, API, and AI service. These are final demo URLs for the project period, not permanent production branding.

Step	Runtime data flow
1	Student logs in from Expo app. AuthProvider stores token through AsyncStorage.
2	AIProvider loads courses, tasks, behavioral logs, and today's schedule from backend API.
3	Student adds or completes tasks. Backend updates StudyTask rows and BehavioralLog rows.
4	Student generates plan. Backend builds payload from subjects, tasks, slots, and logs.
5	If useAi is false, backend rule based scheduler returns a plan directly.
6	If useAi is true, backend sends the payload to FastAPI AI service and stores the response.

7	Frontend renders the plan in Calendar and Home, then Focus can start from a slot or task.
---	---

4.3 DATABASE DESIGN

4.3.1 Database Overview

The database design follows the user-owned data model. Every course, task, slot, schedule, focus session, and behavior log belongs to one user. This is necessary because each student has different courses and different study behavior.

4.3.2 Narrative Data Description

A user creates an account and then completes onboarding. Onboarding creates at least one course and one available slot. A course can own many tasks. A focus session can belong to a course and optionally to a task. When a task is completed, the system stores the actual minutes and updates behavioral logs.

A generated schedule stores both request and response payloads as JSON. This is useful because the plan can be viewed later and because slot statuses can be saved. A slot can be completed or snoozed later from the calendar/focus flow.

4.3.3 Entity Relationship Diagram (ERD)

Relationship	Meaning
User 1 - N Subject	One student has many courses.
Subject 1 - N StudyTask	A course owns many study tasks.
User 1 - N AvailableSlot	A student defines many study time blocks.
User 1 - N FocusSession	Each timer session belongs to the student.
StudyTask 0/1 - N FocusSession	A session can be attached to a task, but it can also be course only.
User 1 - N AiSchedule	A student can generate many daily plans.
User 1 - N BehavioralLog	Daily study hours, focus ratings, and snoozes are stored per day.

4.3.4 Data Integrity and Constraints

- Subjects have unique names per user.
- Task difficulty is between 1 and 10.
- Course difficulty is between 1 and 10 and priority is between 1 and 3.
- Focus rating is between 1 and 5.
- Available slot end time must be after start time.
- A new user cannot finish onboarding without at least one course and one time slot.

4.4 HUMAN-MACHINE INTERFACE

The human-machine interface was the biggest final improvement. The app should feel useful and emotionally friendly for a student. It uses large cards, readable headings, bottom navigation, and short messages. It avoids complicated analytics language.

The following screenshots were taken from the deployed final application using a real demo account. The account contains courses, tasks, completed tasks, a focus session, a generated daily program, and Study Pulse analytics.

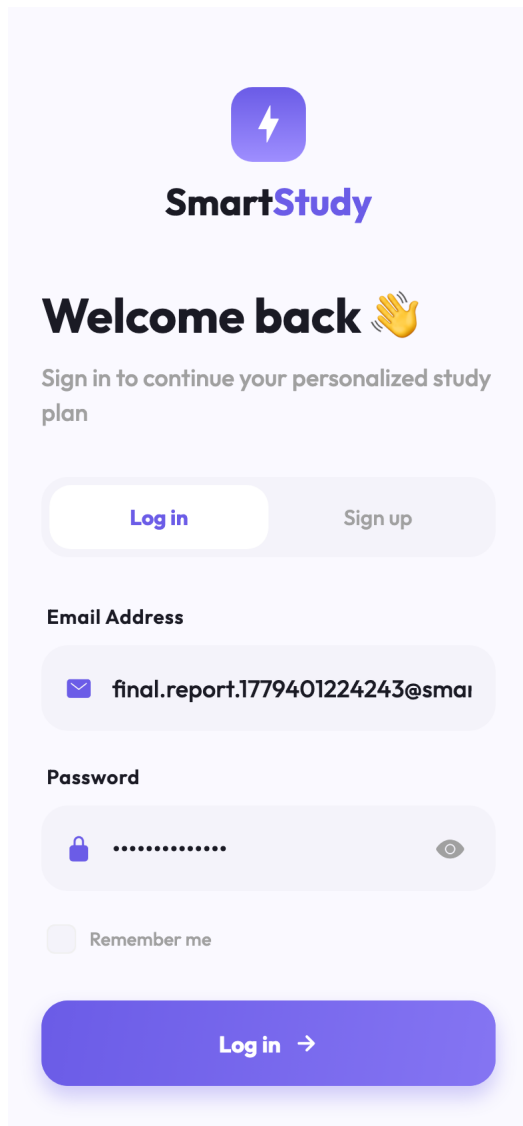


Figure 1. Login with demo email and masked password.

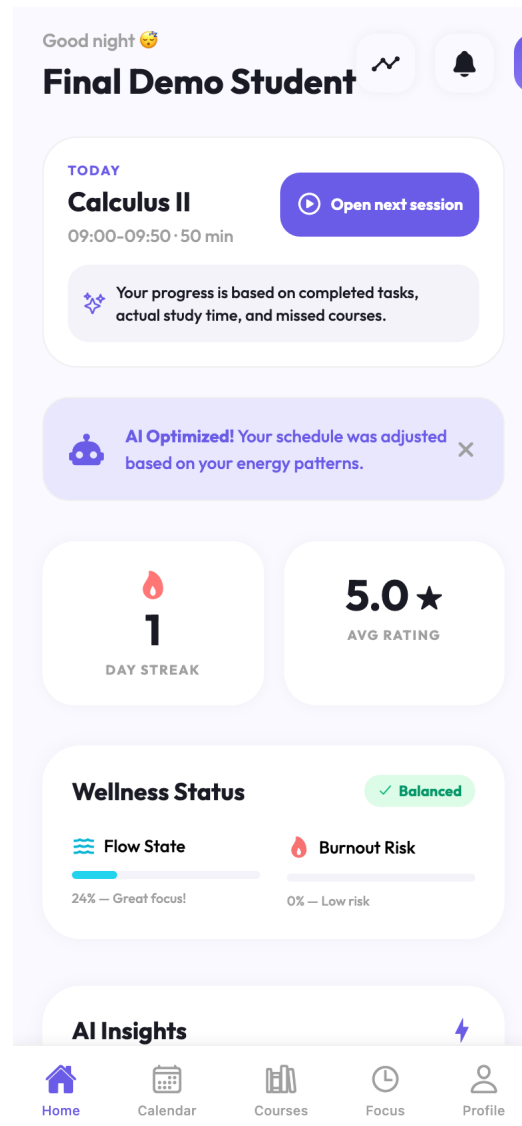


Figure 2. Home dashboard with today's next session and wellness.

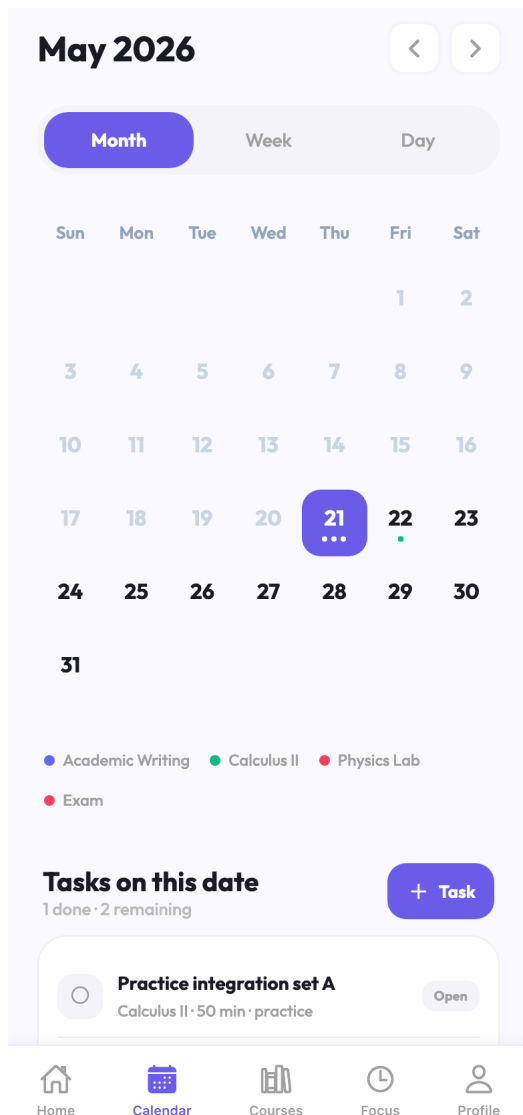


Figure 3. Calendar selected day with done and remaining tasks.

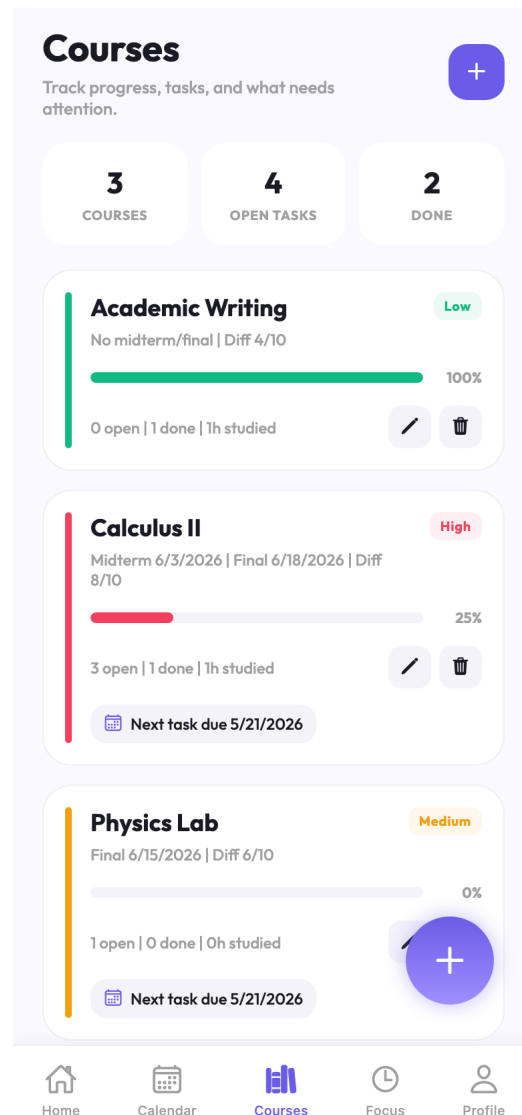


Figure 4. Courses overview with open/done counters.

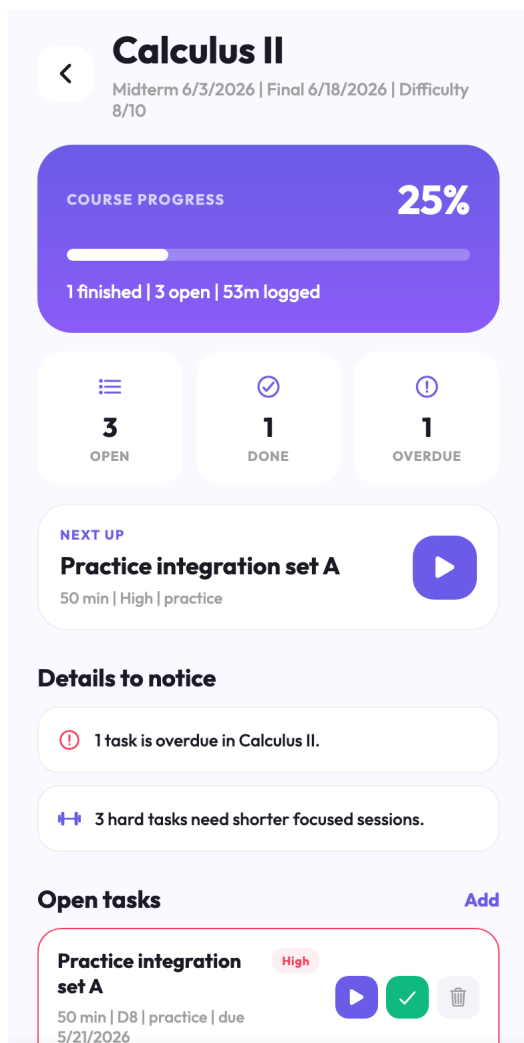


Figure 5. Course detail for Calculus II with midterm and final dates.

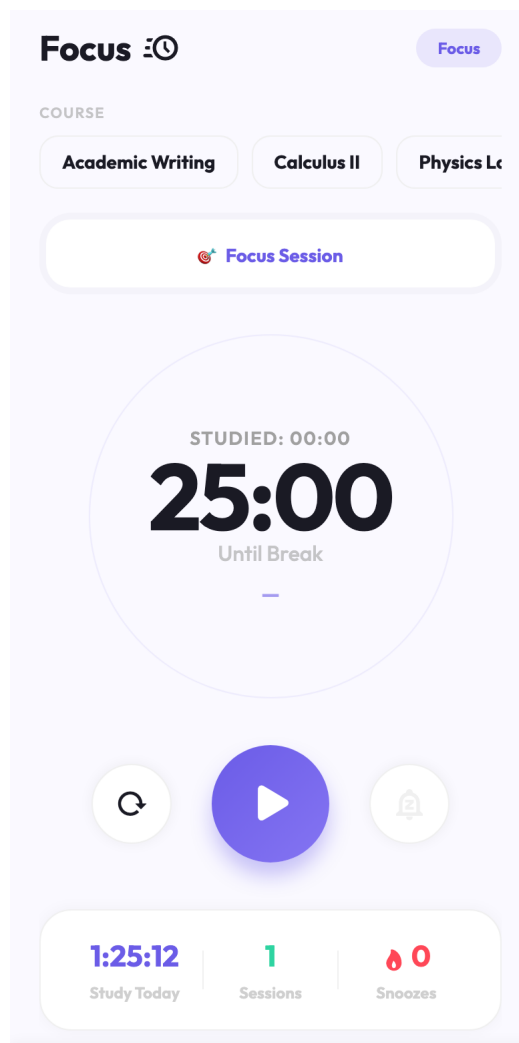


Figure 6. Focus timer for selecting course and task.

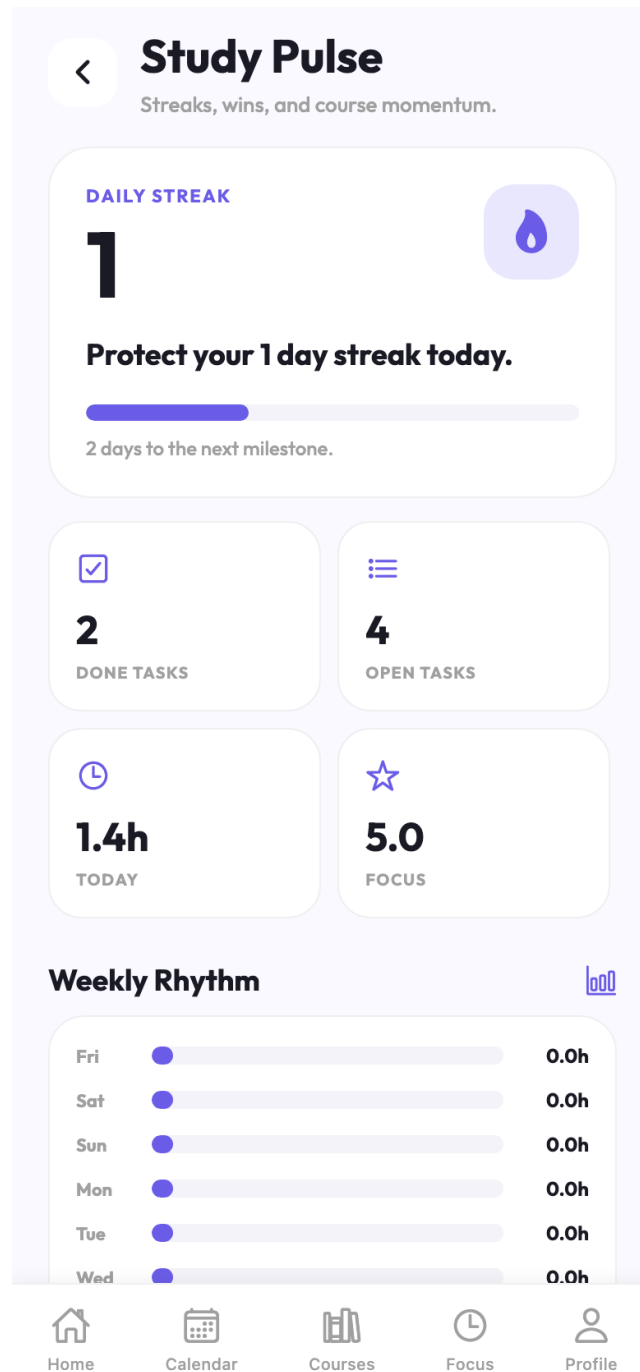


Figure 7. Study Pulse analytics with streak, achievements, and course momentum.

4.4.1 Functional Usage Description

The student logs in, sees the home page, and can immediately understand what is next. If a schedule exists, Home shows the next block. If not, it asks the student to create a plan. Calendar shows tasks by selected date and the daily program. Courses shows academic progress by course. Focus records actual studying. Study Pulse gives a quick emotional and measurable summary.

4.4.2 Inputs

- Email, password, display name, and optional profile changes.

- Course name, difficulty, priority, optional midterm date, optional final date.
- Task title, course, duration, difficulty, priority, due date, and type/tag.
- Available study slots with start and end time.
- Focus session rating, session duration, and whether the task was finished.

4.4.3 Outputs

- Daily plan with time slots, study blocks, breaks, and task references.
- Course progress percentage, open tasks, finished tasks, and actual minutes.
- Calendar indicators for tasks and exam dates.
- Study Pulse metrics including streak, done tasks, open tasks, focus, and weekly rhythm.
- Small encouragement messages when a day has completed tasks.

4.5 DETAILED DESIGN

4.5.1 Software Detailed Design

The front end uses context providers for authentication, AI/data loading, focus state, theme, and navigation. This keeps screen components smaller and makes shared state available across screens. For example, the Focus provider can update session time while Home reads the same study hours.

The backend uses controllers for HTTP endpoints and application services for business logic. The SubjectService handles course creation and duplicate checks. The TaskService handles task completion and behavioral minute updates. The ScheduleService decides whether to use AI or the backend rule based scheduler.

The AI engine receives an IncomingPayload. It groups recent tasks by subject, calculates burnout score, calculates difficulty factors, and then asks Gemini to generate structured schedule output. If Gemini is not available or the call fails, the heuristic fallback produces 50/10 study and break slots.

Frontend part	Design role
AuthProvider	Loads stored tokens, login/register, refreshes current user, and handles auth failure.
AIProvider	Loads subjects, tasks, behavior logs, schedule, and exposes create/complete/generate functions.
FocusProvider	Keeps active session state, timer state, slot status, and partial session switching.
NavigationProvider	Controls bottom tabs and internal pages like course detail and analytics.
ThemeProvider	Provides light colors, fonts, and common visual language for all screens.
Screen	Final purpose

LoginScreen	Email/password login and signup with password validation.
OnboardingScreen	First course, optional exams, first task, and study time slots.
DashboardScreen	Today's next action, streak, rating, wellness, and quick analytics entry.
CalendarScreen	Month/week/day view, tasks by date, task creation, and daily program.
TasksScreen	Courses overview, progress, duplicate course handling, and date editing.
CourseDetailScreen	Per-course progress, open tasks, finished work, details to notice.
FocusScreen	Timer, course/task selection, finishing session, rating, and course switching.
AnalyticsScreen	Study Pulse metrics, achievements, weekly rhythm, and course momentum.

5. IMPLEMENTATION

5.1 Overview of the AI Engine Implementation

The AI Engine is implemented as a FastAPI service named Smart Study Planner - AI Engine. It exposes `/api/v1/optimize-schedule` and `/api/v1/analytics/performance/{user_id}`. The main schedule endpoint receives user history, subjects, available slots, deadline, and current tasks.

The AI implementation is not only a prompt. Before calling Gemini, the service calculates analysis results. The final response contains status, analysis_results, and ai_schedule. This makes the backend able to store both the decision context and the final schedule.

5.2 Core Components and Logic

5.2.1 Server Initialization and Routing

The backend Program.cs registers controllers, JSON converters for DateOnly and TimeOnly, FluentValidation, infrastructure services, application services, JWT authentication, CORS, and health checks. The health endpoint checks both the database and the AI service.

The AI main.py creates a FastAPI app and includes the API router with prefix `/api/v1`. This separation is simple but clear. The final Docker Compose file runs the AI service next to the backend and exposes it only through Caddy and the internal network.

Backend service	Important final behavior
AuthService	Creates Identity users, checks password, issues access and refresh tokens, revokes token chain.
UserService	Completes onboarding and creates initial courses, tasks, and study slots.
SubjectService	Creates/updates courses with priority, difficulty, midterm, final, and duplicate checks.
TaskService	Creates tasks, completes tasks, records study minutes, snoozes tasks, and sends notifications when possible.
FocusSessionService	Starts and completes focus sessions, stores duration/rating, and updates task actual minutes.
ScheduleService	Builds rule based plans, calls AI when selected, persists schedule payloads, and updates slot statuses.
AnalyticsService	Calculates streak, average focus, snooze rate, completed task count, planning error, badges, and weekly hours.

5.2.2 Intelligence Layer (Gemini Integration)

The Gemini integration builds a detailed prompt with burnout risk, per-subject difficulty factors, available slots, and current tasks. The model is asked to return JSON containing scheduled_slots, postponed_tasks, and ai_message. The prompt also asks for breaks and for not ending the plan with a break.

The intelligence layer has a fallback. If the Gemini call fails or returns invalid data, `generate_heuristic_fallback` creates a basic 50/10 style schedule. Also, the backend itself can generate a plan without AI when `useAi` is false. This double fallback is important because the final app should not be blocked by the model.

The backend rule based scheduler scores every planner item before placing it inside an available time slot. Priority gives strong weight, difficulty adds more weight, old neglected tasks receive stale score, and near deadlines add due score. The selected task is placed for up to 50 minutes, then a 10 minute break is inserted if there is enough remaining time.

This logic is intentionally explainable. A student may not see the formula, but the result is predictable: high priority, hard, urgent, and ignored tasks appear earlier. This is better for the final project because it can be defended in front of a reviewer and it still makes sense without external AI.

AI / rule signal	How it affects the plan
Priority	High priority tasks receive more score and appear earlier.
Difficulty	Hard courses and hard tasks receive more study attention.
Deadline	Near or overdue work gets a stronger due score.
Days since last study	Courses that were ignored receive stale score.
Burnout score	AI mode can reduce heavy work and add rest when exhaustion is high.
Difficulty factor	AI mode adjusts duration when actual time is usually longer than estimated.

5.3 Functional Proof and End-to-End Testing

The final demo account was created through the backend API and then opened from the deployed frontend. The demo contains three courses: Calculus II, Physics Lab, and Academic Writing. Calculus has both midterm and final dates. Physics has only a final date. Academic Writing has no midterm or final, which proves that the no-exam course case works.

Six tasks were created. Two tasks were completed. One focus session was started and completed with a 5/5 focus rating. A schedule was generated without AI first, and the daily program showed eight blocks including study sessions and breaks. This confirms the main final flow from backend to frontend.

Demo object	Data used in final screenshots
User	final.report.1779401224243@smartstudy.test
Course 1	Calculus II - difficulty 8/10, high priority, midterm 2026-06-03, final 2026-06-18.
Course 2	Physics Lab - difficulty 6/10, medium priority, final 2026-06-15, no midterm.
Course 3	Academic Writing - difficulty 4/10, low priority, no midterm and no final.

Completed tasks	Midterm formula sheet and Draft introduction paragraph.
Focus session	25 minutes on Calculus II with focus rating 5/5.
Generated plan	May 21, 2026 daily program from 09:00 to 20:25 with breaks.

Test item	Final result
Register/login	Demo email logged in successfully using JWT authentication.
Onboarding	Required course and study slots were accepted.
Course dates	Both dates, final only, and no dates were all represented.
Tasks	Open and completed tasks appeared correctly in Calendar and Courses.
Focus	A focus session recorded 25 minutes and a 5/5 rating.
Schedule	Rule based daily program generated 8 slots for May 21, 2026.
Analytics	Study Pulse showed 1 day streak, 2 done tasks, 4 open tasks, and 1.4h today.

5.4 Implementation Robustness

The final implementation includes validations at several levels. The front end checks signup password strength. The backend validators check emails, passwords, task duration, focus rating, course difficulty, priority, and available slot logic. This makes wrong inputs less likely to enter the system.

The duplicate course case was improved. If the user tries to add a course that already exists, the frontend refreshes the list and tells the student that the course is already in the courses. This is better than leaving the user with a confusing message.

The deployment is also robust enough for a final demo. GitHub Actions can sync the source to `/TMP/smart-study-planner/source` and run `deploy-remote.sh`. The script builds Docker images, starts containers, validates Caddy configuration, and reloads it. If later the temporary project is removed, it is separated from the main Lisan website.

6. RESULT & CONCLUSION

6.1 Results and Discussion

The final result is a working application with a clear student flow. A student can register, set up courses, add tasks, generate a plan, study with the focus timer, and then see analytics. The result is more complete than the mid term because the final version connects the whole loop from planning to doing to reflection.

The design result is also stronger. Courses now feel like real academic courses, not generic subjects only. Midterm and final dates are optional and visible. Calendar tasks show done and remaining work. Study Pulse shows progress in a light way. The student can feel that the app is helping and not only ordering him around.

6.1.1 Testing and Validation

Testing was done through API calls, deployed web screenshots, and a final health check. The backend health returned Healthy for database and AI service. The demo user data was created through the public API, then the UI was opened from the public frontend URL to verify that the screens displayed the same data.

The screenshots in section 4.4 are evidence of end-to-end behavior. They show login, dashboard, calendar, courses, course detail, focus, and Study Pulse. This is important because the final project is not only backend endpoints. The user-facing result was checked visually too.

6.1.2 What was Learned

The biggest lesson is that a study planner becomes useful only when the workflow is simple. If the student must fill too many forms before seeing value, the app feels heavy. For that reason, the final flow lets the student start with one course and one slot, then improve later.

Another lesson is that AI should not be the only engine. The rule based plan gives stable behavior. The AI option can improve the plan, but the student still has a working application without it. This is a better final product decision because reliability comes first.

The last lesson is that analytics should feel human. Numbers like streak, focus, done tasks, and course momentum are easy to understand. Large complex charts can look smart but make the student more confused. The final analytics became smaller, but more useful.

6.2 Conclusion

Smart Study Planner was completed as a functional multi-service application. It includes a React Native/Expo front end, ASP.NET Core backend, SQLite database deployment, FastAPI AI service, Gemini integration, Docker deployment, and GitHub Actions deployment workflow.

The project solved the original problem by connecting planning and real behavior. It can plan a day, record what the student actually studied, and show progress by course. It can also handle partial study sessions and course switching, which is close to how students actually study in real life.

The final version is not perfect, but it is complete enough to be used and demonstrated. It shows the main concept clearly: a planner that does not only store tasks, but tries to understand how the student is moving through the semester.

6.3 Future Work and Improvements

- Improve the calendar creation flow with multi-day task creation for selecting four or five days at once.
- Add stronger per-course analytics, such as time spent, upcoming exams, hard tasks, and missed study days.
- Add richer onboarding suggestions for university midterm and final dates, but keep manual control.
- Improve real device testing so the touch and keyboard behavior stays smooth on different phone sizes.
- Move temporary SQLite deployment to a production database if the app becomes permanent.
- Add real email delivery for password reset instead of the current backend token stub.
- Continue improving the AI schedule prompt and make the AI explanation shorter and more student friendly.

REFERENCES

- Kahneman, D., & Tversky, A. Planning fallacy and decision making under uncertainty.
- Maslach, C., Schaufeli, W. B., & Leiter, M. P. Job burnout and academic exhaustion concepts.
- Baker, R. S., & Inventado, P. S. Educational Data Mining and Learning Analytics.
- Microsoft Documentation. ASP.NET Core, ASP.NET Identity, JWT authentication, and Entity Framework Core.
- React Native and Expo Documentation. Cross-platform mobile and web application development.
- FastAPI Documentation. Python API framework for high performance services.
- scikit-learn Documentation. Linear regression, logistic regression, and model persistence utilities.
- Google Gemini API Documentation. Generative model integration and structured JSON output.
- Docker and Caddy Documentation. Containerized services, reverse proxy, and automatic HTTPS.
- Project source code: Smart Study Planner frontend, backend, AI engine, and deployment scripts.